

Técnica de Diseño Multiparadigma FP/OO — Referencia Rápida

1 — TABLA DE DECISIÓN OO / FUNCIONAL (TABLA 4.1)

Característica del problema	Enfoque recomendado	Justificación
Entidad con identidad y estado mutable	OO (clases con encapsulación)	Encapsulación protege la integridad del estado
Flujo de transformación de datos (ETL)	Funcional (pipelines)	Modularidad por composición de funciones
Comportamiento parametrizable simple	Funcional (lambdas)	Configurabilidad ligera sin sobrecarga de clases
Comportamiento parametrizable complejo	OO (interfaces / Strategy)	Polimorfismo con estado e implementaciones intercambiables
Nuevos tipos de datos frecuentes	OO (jerarquías abiertas de subtipos)	Extensibilidad vía polimorfismo de subtipo
Nuevas operaciones frecuentes	Funcional (ADT sellado + pattern matching, HOF)	Extensibilidad vía nuevas funciones sobre tipos cerrados
Nuevos tipos Y nuevas operaciones simultáneos	Híbrido (álgebra de objetos / tagless final)	Extensibilidad bidimensional — resuelve el Expression Problem
Alta concurrencia	Funcional (inmutabilidad)	Reducción de efectos colaterales y bloqueos

2 — LOS 6 PASOS DE LA TÉCNICA

1. Descomposición y clasificación

Descomponer los requisitos en componentes candidatos y aplicar la tabla de decisión a cada uno.

Salida: lista de componentes clasificados como entidad mutable (OO) o valor/función (FP).

2. Detección de patrones existentes

Inventariar los patrones de diseño presentes (Strategy, Visitor, Observer, etc.) usando el catálogo.

Cada patrón detectado es candidato de reformulación con su mecanismo M1–M5.

3. Definición de interfaces híbridas

Diseñar las interfaces de las clases OO principales. En puntos de variación, sustituir interfaces de un solo método por parámetros de tipo función.

4. Modelado visual

Construir diagramas de clases y secuencia con el perfil UML ligero, marcando los estereotipos: «pure», «ValueObject», «signal», «var».

5. Refactorización hacia inmutabilidad

Revisar el diseño buscando reducir estado mutable compartido. Si una clase tiene mayoría de métodos «pure», evaluar transformarla en módulo funcional o ValueObject.

6. Validación O(1) vs O(N)

Declarar los pasos de evolución anticipados y verificar que las decisiones tomadas los resuelven con costo acotado.

3 — MECANISMOS M1–M5

Mec.	Nombre	Pregunta clave
M1	Álgebras de objetos / Tagless Final	¿Cómo crezco tipos y operaciones simultáneamente sin tocar lo existente?
M2	Type Classes / Generalized Interfaces	¿Cómo le doy comportamiento nuevo a tipos que ya existen sin modificarlos?
M3	ADT sellado + Pattern Matching	¿Cómo descompongo por casos sin el andamiaje accept/visit del Visitor?
M4	HOF + Closures + Composición FP	¿Cómo represento comportamiento puro sin jerarquías de clases?
M5	Signals / Actores / Continuaciones	¿Cómo propago cambios sin acoplar sujetos a observadores?

4 — ESTEREOTIPOS UML DEL PERFIL LIGERO

«pure» — método sin efectos de lado, referencial y transparente

«ValueObject» — clase inmutable que representa un valor (no entidad)

«signal» — atributo reactivo / observable

«var» — atributo mutable con estado explícito

{immutable} — restricción: los campos no cambian tras la construcción